Labreport Advanced Computer Architecture
Lab 1

Jonas Roels & Niels Van Bossche

February 12, 2024



Class: MEIsw

Contents

1 Introduction **1**

2 Copying between CPU and GPU memory **1**

3 Thread and block size **1**

1 Introduction

In this lab we will get to know GPU programming using the Nvidia CUDA C Compiler (NVCC). The point of this lab is to develop code that inverses an array. This can easily be done on the GPU using the code found in the listing below [1]. This function takes the input and output array as well as the size of said array as parameters. It then attempts to swap the number at a certain index. This index is calculated using the block index, block size and thread index.

```
__global__ void swap(int* in, int* out, int size) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if(i < size) out[size-1-i] = in[i];
}
```

2 Copying between CPU and GPU memory

Initially an array of sequential integers is initialized by the CPU in its memory. To use this during computations on the GPU it needs to be copied to its memory space. This can be done using:

```
cudaMemcpy(destination_on_GPU, source_on_CPU, size*sizeof(int), cudaMemcpyHostToDevice);
```

After the calculations are done we need to copy it back to the CPU memory space so we can print the results. This can be achieved using:

```
cudaMemcpy(out, output, size*sizeof(int), cudaMemcpyDeviceToHost);
```

Note that these memory locations on the GPU need to be allocated using:

```
cudaMalloc(&output, size*sizeof(int));
```

3 Thread and block size

Because one block on the GPU can't contain more then 1024 threads, flipping an array of more then 1024 integers needs to be split up in more blocks. To determine the number of blocks and corresponding thread count, you can use the following code:

```
int blocks = size/1024 + 1;
int threads = size / blocks + 1;
```

The number of blocks is the minimum amount we can use since we divide is by 1024. To spread the number of threads as evenly as possible we divide the total number of integers in the array by the blocks we use.

[1] The full source code can be found here: https://github.com/RoelsJonas/Advanced-Computer-Archticture-Lab-1